

Task 1:

What are streams?

3min - 09.56 to 10.00

In programming, a **stream** is a **sequence of data elements** made available over time. It's a concept used to handle **continuous flows of data** efficiently, especially when you don't want or can't load all data into memory at once.

Task 2:

Write about :

Boilerplate code:

Lack of parallelism

Lack of Composition

Boilerplate Code

Boilerplate code refers to **repetitive and standardized code** that must be written in many places with little to no variation. It often doesn't contribute to core logic but is necessary due to language or framework requirements.

Eg

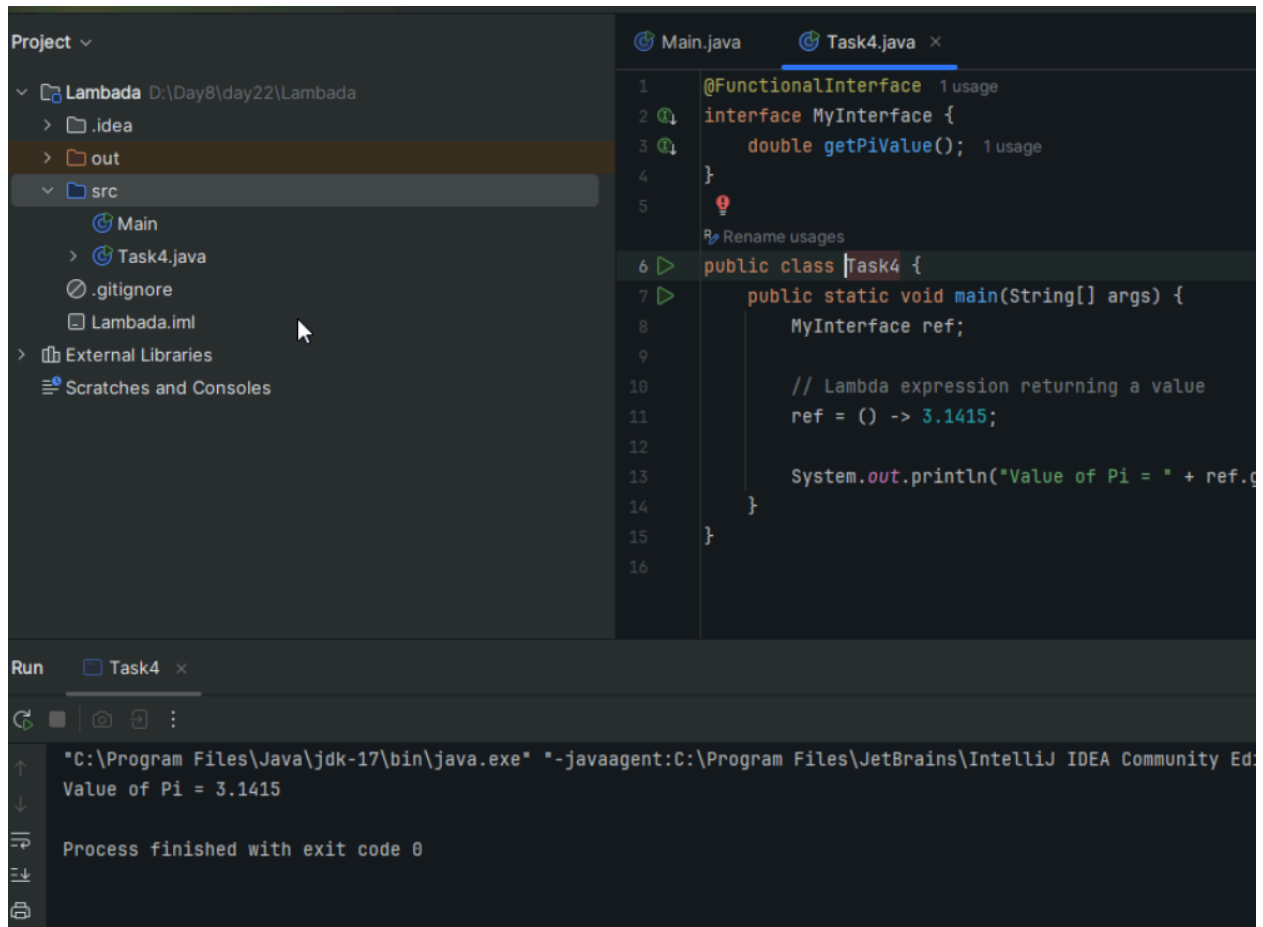
```
public class User {  
    private String name;  
    private int age;  
  
    public User(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    // Getters and Setters for each field  
    // toString(), equals(), hashCode(), etc.  
}
```

Task 3:

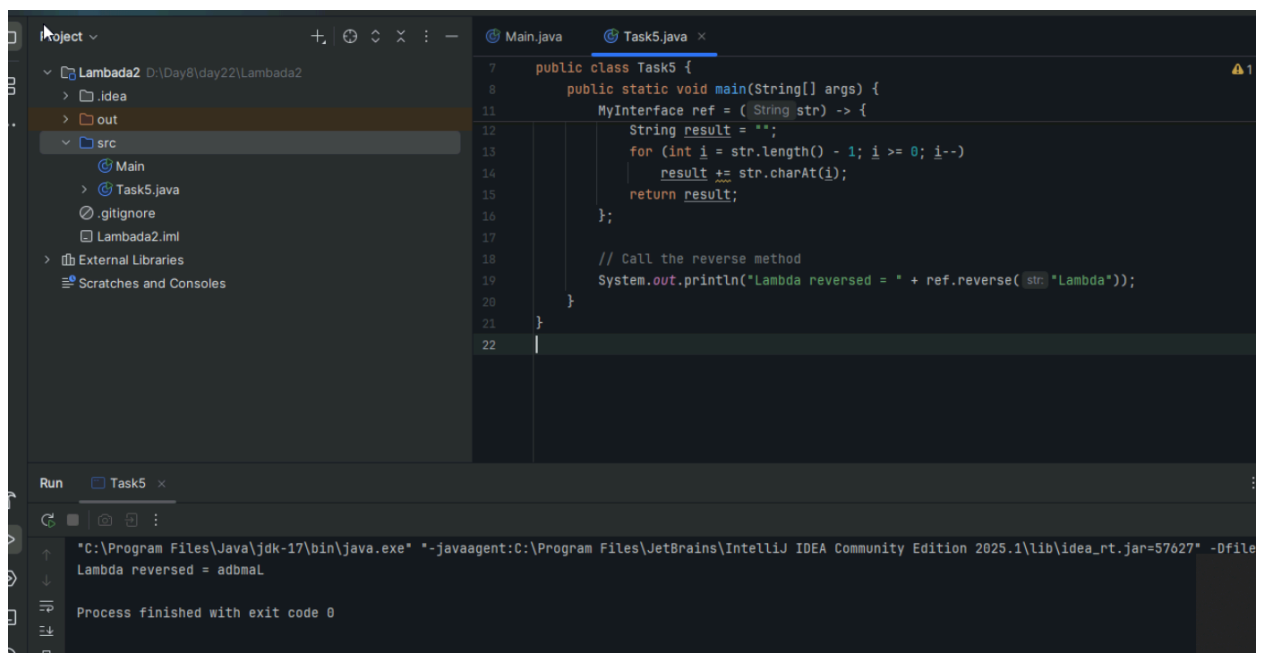
List of Intermediate and terminal operations

These operations return a **new stream**. They are **lazy**, meaning they don't execute until a **terminal operation** is invoked.

Task 4:



Task 5:



Task 6:

```
Task6.java
import java.util.ArrayList;
import java.util.List;

public class Task6 {

    // Create an object of list using ArrayList
    static List<String> places = new ArrayList<>(); 6 usages

    // Preparing our data
    public static List<String> getPlaces() { 1 usage

        // Add places and country to the list
        places.add("Nepal, Kathmandu");
        places.add("Nepal, Pokhara");
        places.add("India, Delhi");
        places.add("USA, New York");
        places.add("Africa, Nigeria");

        return places;
    }

    public static void main(String[] args) {

        List<String> myPlaces = getPlaces();
        System.out.println("Places from Nepal:");

        // Filter places from Nepal
        myPlaces.stream()
```

```
21
22 ▶ public static void main(String[] args) {
23
24     List<String> myPlaces = getPlaces();
25     System.out.println("Places from Nepal:");
26
27     // Filter places from Nepal
28     myPlaces.stream()
29         .filter((String p) -> p.startsWith("Nepal"))
30         .map((String p) -> p.toUpperCase())
31         .sorted()
32         .forEach((String p) -> System.out.println(p));
33 }
34 }
35 |
```

```
C:\Program Files\Java\jdk-17\bin\ja
Places from Nepal:
NEPAL, KATHMANDU
NEPAL, POKHARA

Process finished with exit code 0

a3 > src > Task6
```

Task 7:

```

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

public class Task7 {

    public static void main(String[] args) {

        // Create a list of 5 integers
        List<Integer> numbers = new ArrayList<>();
        numbers.add(2);
        numbers.add(4);
        numbers.add(6);
        numbers.add(8);
        numbers.add(10);

        // Use stream to calculate square of each number
        List<Integer> squareOfNums = numbers.stream()
            .map(Integer num -> num * num)
            .collect(Collectors.toList());

        // Display the squared numbers
        System.out.println("Original Numbers: " + numbers);
        System.out.println("Squares: " + squareOfNums);
    }
}

```

```

"C:\Program Files\Java\jdk-17\bin\java.exe"
Original Numbers: [2, 4, 6, 8, 10]
Squares: [4, 16, 36, 64, 100]

Process finished with exit code 0

```

Task 8:

What do you understand by map()?

In programming (especially in functional programming and stream APIs like Java or JavaScript), **map()** is a method used to **transform** each element of a collection or stream by applying a **function** to it.

map() **does not change the original elements** — it creates a **new collection or stream** with **transformed** elements.

Task 9:

Write a code to create an array list and filter the values which are odd numbers and display them..

Hint:

```
List<Integer> addNumbers = numbers.stream()  
    .filter(num -> num % 2 !=0)  
    .collect(Collectors.toList());
```

Main.java

Task9.java ×

```
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

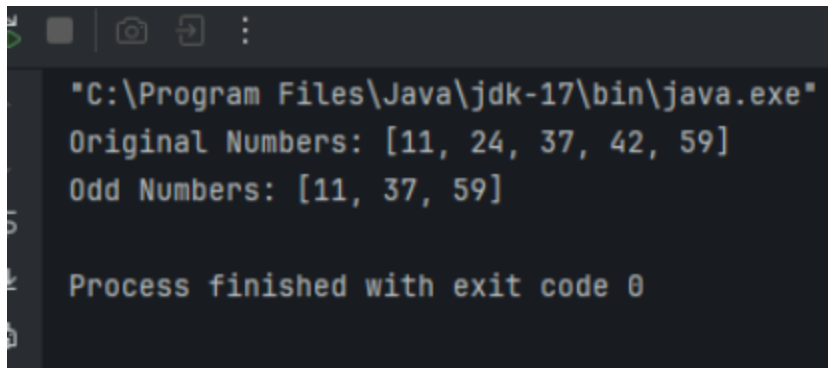
public class Task9 {

    public static void main(String[] args) {

        // Create an ArrayList of integers
        List<Integer> numbers = new ArrayList<>();
        numbers.add(11);
        numbers.add(24);
        numbers.add(37);
        numbers.add(42);
        numbers.add(59);

        // Filter odd numbers using streams
        List<Integer> oddNumbers = numbers.stream()
            .filter(Integer num -> num % 2 != 0)
            .collect(Collectors.toList());

        // Display the original and filtered list
        System.out.println("Original Numbers: " + numbers);
        System.out.println("Odd Numbers: " + oddNumbers);
    }
}
```



```
"C:\Program Files\Java\jdk-17\bin\java.exe"  
Original Numbers: [11, 24, 37, 42, 59]  
Odd Numbers: [11, 37, 59]  
  
Process finished with exit code 0
```

Task 10:

What do you understand by filter()?

In programming (especially in **functional programming** and **stream APIs** like Java, JavaScript, or Python), the `filter()` method is used to **select elements** from a collection **that match a condition**.

`filter()` removes unwanted elements by **applying a condition (predicate)** to each item. Only items that return `true` are kept.

Task 11:

Wap to create an array list to remove duplicate values from the List.


```

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

public class Task11 {

    public static void main(String[] args) {

        // Create an ArrayList with duplicate integers
        List<Integer> numbers = new ArrayList<>();
        numbers.add(10);
        numbers.add(20);
        numbers.add(10);
        numbers.add(30);
        numbers.add(20);
        numbers.add(40);

        System.out.println("Original List with duplicates: " + numbers);

        // Remove duplicates using distinct()
        List<Integer> noDuplicates = numbers.stream()
            .distinct()
            .collect(Collectors.toList());

        System.out.println("List after removing duplicates: " + noDuplicates);
    }
}

```

```

"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Progr
Original List with duplicates: [10, 20, 10, 30, 20, 40]
List after removing duplicates: [10, 20, 30, 40]

Process finished with exit code 0
|

```

Task12:

What do you understand by distinct()?

In programming (especially in **Java Streams**), the `distinct()` method is used to **remove duplicate elements** from a stream or collection, returning **only unique elements**.

`distinct()` filters out duplicate values based on **object equality** (using `equals()` and `hashCode()` in Java).

Task 13:

Wao to create an arrayList of your friends using string and try to sort them and display

```
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

public class Task13 {

    public static void main(String[] args) {

        // Create an ArrayList of friend names
        List<String> friends = new ArrayList<>();
        friends.add("Meera");
        friends.add("Anjali");
        friends.add("Ravi");
        friends.add("Zara");
        friends.add("Bhavna");

        System.out.println("Original List: " + friends);

        // Sort the names using stream
        List<String> sortedNames = friends.stream()
            .sorted()
            .collect(Collectors.toList());

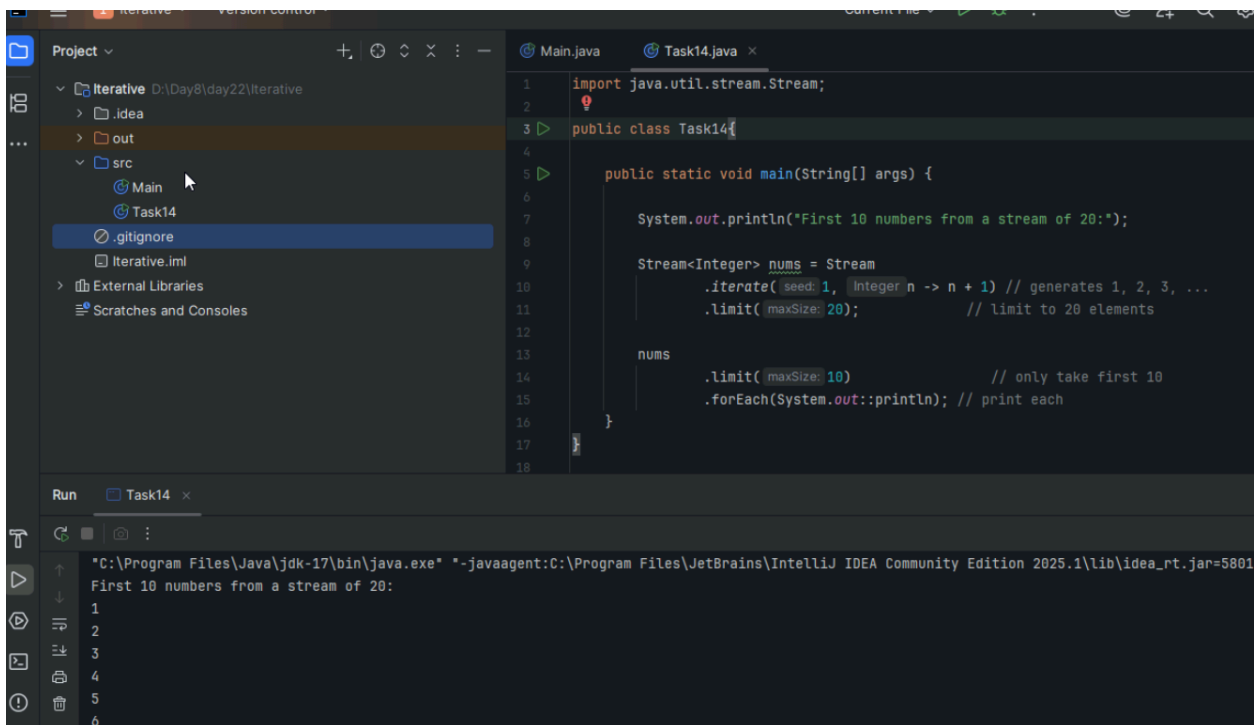
        System.out.println("Sorted List: " + sortedNames);
    }
}
```

```
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent
Original List: [Meera, Anjali, Ravi, Zara, Bhavna]
Sorted List: [Anjali, Bhavna, Meera, Ravi, Zara]

Process finished with exit code 0
```

Task 14:

Wap to run a loop / iterate() and limit it to 20 values (1 to 2)
While displaying use for each to limit till 10 numbers.



The screenshot shows the IntelliJ IDEA IDE with a project named 'Iterative'. The 'src' folder contains 'Main.java' and 'Task14.java'. The 'Task14.java' file is open, showing the following code:

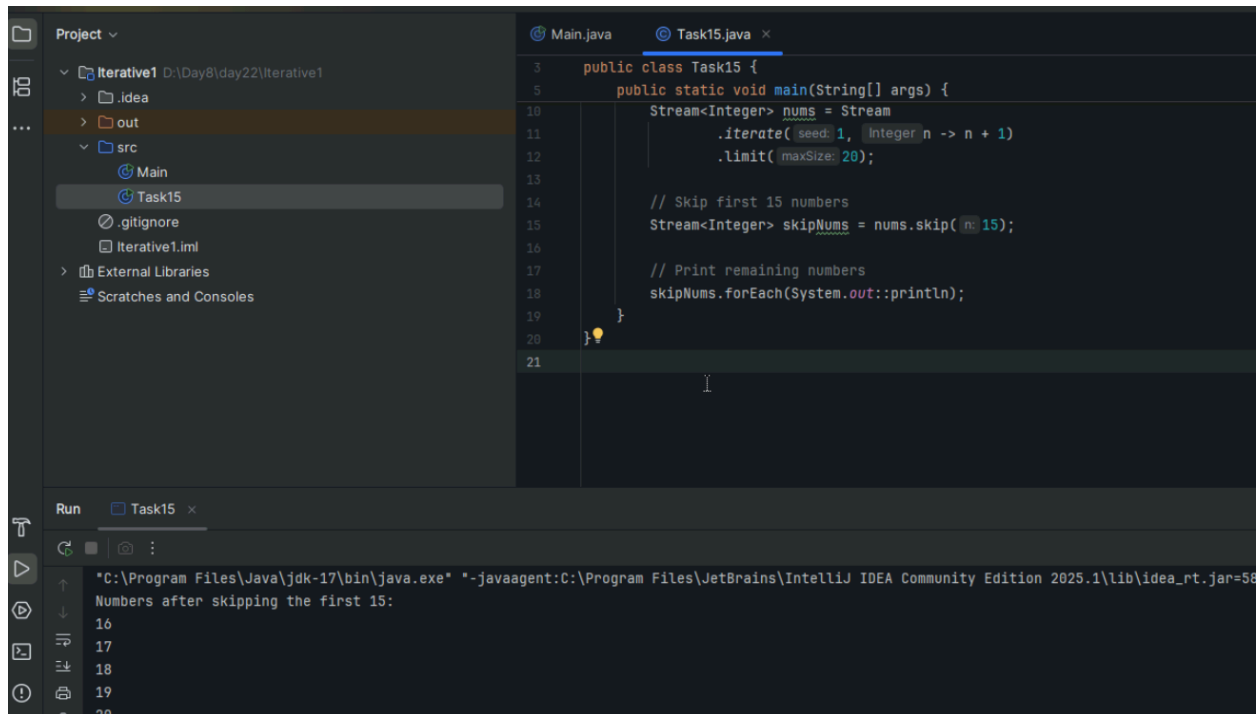
```
1 import java.util.stream.Stream;
2
3 public class Task14{
4
5     public static void main(String[] args) {
6
7         System.out.println("First 10 numbers from a stream of 20:");
8
9         Stream<Integer> nums = Stream
10             .iterate(1, Integer n -> n + 1) // generates 1, 2, 3, ...
11             .limit(maxSize: 20);           // limit to 20 elements
12
13         nums
14             .limit(maxSize: 10)           // only take first 10
15             .forEach(System.out::println); // print each
16     }
17
18 }
```

The 'Run' tab at the bottom shows the execution output:

```
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.1\lib\idea_rt.jar=5801
First 10 numbers from a stream of 20:
1
2
3
4
5
6
```

Task 15: (similar to Task 14)

Wap to create an array List skip 15 numbers and print the output using foreach loop



Task 16:

Explain limit and skip methods..

Both `limit()` and `skip()` are **intermediate operations** used in stream processing (e.g., Java Streams). They help control which elements flow through the stream.

limit(n)

What it does:

- Returns a stream consisting of the **first n elements** from the original stream.
- If the stream has fewer than n elements, it returns all elements.

Use Case:

- Useful when you want to **process only a subset** (e.g., the top 5 records).

Example:

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);
```

```
List<Integer> limited = numbers.stream()  
    .limit(3)  
    .collect(Collectors.toList());
```

```
System.out.println(limited); // Output: [1, 2, 3]
```

skip(n)

What it does:

- Returns a stream that **skips the first *n* elements** and processes the rest.
- If the stream has fewer than *n* elements, returns an empty stream.

Use Case:

- Useful for **pagination**, e.g., skipping the first page's elements.

Example:

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);
```

```
List<Integer> skipped = numbers.stream()  
    .skip(3)  
    .collect(Collectors.toList());
```

```
System.out.println(skipped); // Output: [4, 5, 6]
```

Combined Example:

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);
```

```
List<Integer> result = numbers.stream()  
    .skip(2)           // Skip first 2 elements -> [3, 4, 5, 6]  
    .limit(2)          // Take next 2 elements -> [3, 4]  
    .collect(Collectors.toList());
```

```
System.out.println(result); // Output: [3, 4]
```

Task 17:

What is the difference between mutable and immutable?

Mutable

- **Mutable objects** are like a notebook where you can **write, erase, and rewrite** anything any time.
 - You can **change their content or state** after creating them.
 - Example: A list where you can add or remove items.
 - Because their data changes, you have to be careful when multiple parts of a program access the same mutable object — it can cause unexpected results.
-

Immutable

- **Immutable objects** are like a printed book — **once printed, you cannot change the words inside**.
- After you create them, their data **cannot be modified**.
- To "change" an immutable object, you actually create a **new object** with the new data.
- This makes immutable objects very **safe** to use in multi-threaded or complex programs since their state can't change unexpectedly.

Task 18:

What are the debugging tools in Java.. list down a few.

1. Eclipse Debugger

- Integrated into Eclipse IDE
- Breakpoints, step execution, watch variables
- Inspect variables and thread states

2. IntelliJ IDEA Debugger

- Powerful debugging features in IntelliJ IDEA IDE
- Evaluate expressions, conditional breakpoints
- Supports remote debugging

3. NetBeans Debugger

- Debugger integrated in NetBeans IDE
- Supports step-by-step execution, breakpoints, variable inspection

4. JDB (Java Debugger)

- Command-line debugger included in JDK
- Useful for lightweight or remote debugging
- Supports breakpoints, stepping, and inspecting variables

5. VisualVM

- Visual monitoring and profiling tool bundled with JDK
- Can attach to running JVMs
- Supports heap dumps, thread analysis, and CPU profiling

6. JConsole

- Java Monitoring and Management Console
- Monitors JVM performance and resource consumption
- Can help detect issues like memory leaks

Task 19:

Error Messages in Java

Compile time and run time

Compile time : grammatical mistakes ... ;, {}, missing the code

Run time error or exceptions

Stack overflow error

Array index out of bounds

IO exception

1. Compile-Time Errors

- These errors occur **when you compile the code**.
- They are mostly **syntax or grammatical mistakes**.
- The compiler detects these before the program runs.

Common Compile-Time Errors:

- Missing semicolon ;
- Missing or mismatched braces { }
- Wrong variable declarations
- Type mismatches
- Misspelled keywords or identifiers

Example:

```
int x = 10 // Missing semicolon - compile-time error
```

2. Run-Time Errors (Exceptions)

- These errors occur **when the program is running**.
- Usually caused by **illegal operations or unexpected conditions**.
- Can cause the program to **crash** unless handled.

Common Run-Time Errors:

NullPointerException

ArrayIndexOutOfBoundsException

StackOverflowError

IOException

Eg

```
// NullPointerException
```

```
String str = null;
```

```
str.length(); // Causes NullPointerException
```

```
// ArrayIndexOutOfBoundsException
```

```
int[] arr = {1, 2, 3};
```

```
int x = arr[5]; // Invalid index
```

```
// StackOverflowError (recursive method without base case)
```

```
void recurse() {
```

```
    recurse();
```

```
}
```

Task 20:

Stack trace.. What will it do?

Identify the error

Locate the code

Analyze the code

Solution also

A **stack trace** is a detailed report of the active stack frames (method calls) at the point where an exception or error occurs during the execution of a program.

What Does a Stack Trace Do?

1. **Identify the Error**

- Shows the **type of exception/error** (e.g., `NullPointerException`, `ArrayIndexOutOfBoundsException`).
- Provides the **error message** giving clues about what went wrong.

2. Locate the Code

- Lists the **sequence of method calls** that led to the error (called the "call stack").
- Shows the **exact line numbers** in the source files where each method was called, especially the line where the error happened.

3. Analyze the Code

- Helps you **trace the flow of execution** backward from the error point.
- Lets you understand how the program arrived at the problematic state.

4. Solution

- By examining the stack trace, you can **pinpoint bugs**, such as null references or invalid array access.
- Helps in **debugging and fixing** the root cause by reviewing the implicated lines and logic.

```
Exception in thread "main" java.lang.NullPointerException
    at com.example.MyClass.myMethod(MyClass.java:25)
    at com.example.MyClass.main(MyClass.java:10)
```